Das ist ein solides Fundament für eine E-Learning-Plattform. Da deine Controller bereits stehen und die Datenbank-Struktur (PostgreSQL-Dialekt) definiert ist, konzentrieren wir uns nun auf einen **Integrationstest-Plan**.

Da viele Tabellen über Foreign Keys verknüpft sind (z. B. `lesson_content` braucht eine `lesson_id`), ist die **Reihenfolge** der Tests entscheidend.

---

1. Test-Strategie & Reihenfolge

Wir testen in vier Phasen, um sicherzustellen, dass die Relationen korrekt aufgelöst werden:

1. **Stammdaten:** User & Lessons (Basis-Entitäten)
2. **Abhängige Daten:** Lesson Content (1:1/n zu Lesson)
3. **Prozessdaten:** User Progress & User Lesson Result (Verknüpfung User + Lesson)

---

2. Der Testplan (Postman)

Phase 1: Basis-Entitäten (User & Lesson)

Test 1: User erstellen

- **Methode:** `POST`
- **URL:** `{{base_url}}/api/users`
- **Body (JSON):**

JSON

```
{
  "username": "LinguinFan",
  "email": "test@tgm.ac.at",
  "password": "securePassword123",
  "role": "USER"
}
```

Test 2: Lesson erstellen

- **Methode:** `POST`

- **URL:** `{{base_url}}/api/lessons`
- **Body (JSON):**

JSON

```
{
  "title": "Spanisch für Anfänger",
  "description": "Lerne die Grundlagen der spanischen Sprache.",
  "difficulty": 1
}
```

Phase 2: Content hinzufügen

Test 3: Lesson Content verknüpfen

- **Methode:** `POST`
- **URL:** `{{base_url}}/api/lesson-contents`
- **Body (JSON):** (Hier die `lesson_id` aus Test 2 verwenden)

JSON

```
{
  "lessonId": 1,
  "content": "Inhalt der ersten Lektion: Hola, Como estas?"
}
```

Phase 3: Fortschritt & Ergebnisse (Logik-Check)

Test 4: Fortschritt speichern

- **Methode:** `POST`
- **URL:** `{{base_url}}/api/user-progress`
- **Body (JSON):**

JSON

```
{
  "userId": 1,
  "lessonId": 1,
  "percentage": 50,
  "status": "IN_PROGRESS"
}
```

Test 5: Abschlussprüfung (Resultat)

- **Methode:** `POST`
- **URL:** `{{base_url}}/api/user-lesson-results`
- **Body (JSON):**

JSON

```
{
  "userId": 1,
  "lessonId": 1,
  "score": 85,
  "passed": true,
  "completedAt": "2026-02-26T14:00:00"
}
```

---

3. Erwartetes Endergebnis (Datenbank-Zustand)

Nachdem du diese Requests in Postman abgeschickt hast, sollte deine Datenbank wie folgt aussehen (Beispiel-Query Ergebnis):

Tabelle `users`

id	username	email	role
1	LinguinFan	test@tgm.ac.at	USER

users

	id	username	email	password	created_at	last_login	role
1	2	LinguinFan	test@tgm.ac.at	sicheresPasswort123	2026-02-19 14:13:10.7581…	<null>	USER
2	3	LinguinMaster	master@tgm.ac.at	geheimesPasswort123	2026-02-19 14:22:38.3239…	<null>	USER
3	7	LinguinFan	test23@tgm.ac.at	securePassword123	2026-02-26 13:30:27.1522…	<null>	USER

Tabelle `user_progress`

id	user_id	lesson_id	percentage	status
15	7	3	50	IN_PROGRESS

users | user_progress

	id	user_id	lesson_id	percentage	status
1	10	3	2	10	STARTED
2	11	3	2	10	STARTED
3	12	3	2	10	STARTED
4	13	3	2	10	STARTED
5	14	3	2	100	COMPLETED
6	15	7	3	50	IN_PROGRESS

Tabelle `user_lesson_result`

id	user_id	lesson_id	score	passed
2	7	3	85	true

users | user_progress | user_lesson_result

Tx: Auto | DDL | CSV

WHERE | ORDER BY

	id	user_id	lesson_id	score	passed	completed_at
1	1	3	2	85	true	2026-02-19 15:00:00.000000
2	2	7	3	85	true	2026-02-26 14:00:00.000000